UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento: Departamento de Electrónica y Telecomunicaciones

Asignatura: FUNDAMENTOS DE PROGRAMACIÓN

NRC: 36466

Apellidos y nombres: Sara Estalana

Deber: Ejercicios de programación y algoritmia

Fecha: 23 de octubre de 2025

Desarrollo de 20 Problemas.

Presentación

Este trabajo reúne veinte ejercicios introductorios de programación. Cada actividad se desarrolla mediante identificación de datos, explicación del procedimiento, pseudocódigo, comprobación con casos de prueba, esquema de flujo y una propuesta de implementación en lenguaje C. La redacción y los ejemplos se prepararon nuevamente, conservando los temas de presentación.

Índice de ejercicios.

1.1 Media de dos valores

1.2 Valor absoluto de x al cubo

1.3 Divisibilidad entre dos enteros

1.4 Pertenencia a un intervalo

1.5 Conversión de segundos

1.6 Estadística de alturas

1.7 Raíz cuadrada entera

1.8 Detección de números primos

1.9 Primos dentro de un rango

1.10 Cálculo del factorial

1.14 Serie de Fibonacci

1.15 Suma de dígitos

1.16 Número capicúa

1.17 Número cubo perfecto

1.18 Año bisiesto

1.19 Número perfecto

1.20 Conversión decimal a binario

1.21 Triángulo de asteriscos

1.22 Dos valores mayores de una secuencia

1.23 Juego de adivinanza

Problema 1.1. Media de dos valores

Leer dos números reales y calcular su promedio

1. Análisis del problema

Descripción: Se suman los dos datos y se divide el total entre dos. No se requiere validar signos.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada

Validación: Se controlan los límites o datos especiales que podrían producir un r.u.

2. Tabla de objetos

Objeto	Identificador	Clase	Tipo
Primer dato	a	variable	real
Segundo dato	b	variable	real
Promedio	media	variable	real
Divisor	2	constante	entero

3. Pseudocódigo.

Inicio leer a, b media ← (a + b) / 2 Escribir media Fin

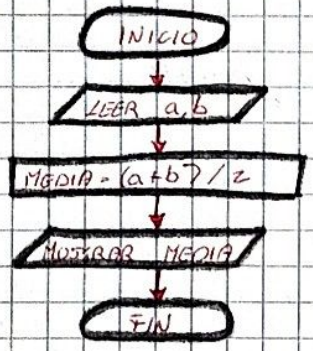
4. Prueba de escritorio.

Entrada / caso	Resultado esperado	
4	6	5
-5	-3	-4
1.5	2.5	2.0

5. Diagrama de flujo

Esquema del flujo: Inicio -> leer a,b -> calcular (a+b)/2 -> mostrar media -> fin

Para dibujarlo, utilicé óvalo para inicio/fin, paralelogramo para entrada/salida, rectangulo para procesos y rombo para decisiones.



6. Código en C (Code::Blocks)

Comprobación: compilar el programa, ejecutarlo y comparar la salida con los casos de prueba de esta sección.

```c
#include <stdio.h>
int main(void) {
    double a, b, media;

    printf("Ingrese dos numeros: ");
    scanf("%lf %lf", &a, &b);

    media = (a + b) / 2.0;
    printf("La media es %.2f\n", media);
    return 0;
}
```

Problema 1.2. Valor absoluto de x al cubo

Leer un real x y mostrar |x| elevado a la tercera potencia

1. Análisis del problema

Descripción: Primero se obtiene la magnitud positiva de x. Después se multiplica ese valor por sí mismo tres veces.

Datos de entrada: Los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: Se comprueban los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos:

Objeto	Identificador	Clase	Tipo
Número ingresado	x	variable	real
Valor absoluto	absx	variable	real
Resultado	resultado	variable	real
Cero	0	constante	entero

Pseudocódigo

Inicio leer x Si x < 0 entonces absx ← -x si no absx ← x resultado ← absx * absx * absx Escribir resultado Fin.

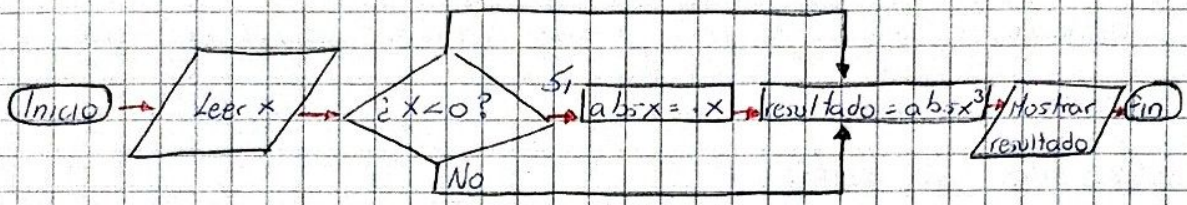
4. Prueba de escritorio

Entrada / caso	Resultado esperado
-3	3 27
2	2 8
-0.5	0.5 0.125

5. Diagrama de flujo.

Esquema del flujo: Inicio → leer x → ¿x<0? → asignar abs=x → elevar al cubo → mostrar → fin.

Para dibujarlo, utiliza óvalo para inicio/fin, paralelogramo para entrada/salida, rectángulo para procesos y rombo para decisiones.



6. Código en C (Code::Blocks)

```c
#include <stdio.h>

int main(void) {
    double x, absx, resultado;

    printf("Ingrese x: ");
    scanf("%lf", &x);

    // Obtiene el valor absoluto de x
    absx = (x < 0) ? -x : x;

    // Eleva el valor absoluto al cubo
    resultado = absx * absx * absx;

    printf("|x|^3 = %.3f\n", resultado);

    return 0;
}
```

Comprobación: compilar el programa, ejecutarlo y comparar la salida de con los casos de prueba de esta sección.

Problema 1.3. Divisible

Determinar si un entero x es divisible exactamente por otro entero y.

1. Análisis del Problema.

Descripción: El residuo de x entre y permite decidir: si es cero, la división es exacta. Antes se comprueba que el divisor no sea cero.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos

Objeto	Identificador	Clase	Tipo
Dividendo	x	variable	entero
Divisor	y	variable	entero
Residuo	r	variable	entero
Cero	0	constante	entero

3. Pseudocódigo.

Inicio leer x, y, si y = 0 entonces escribir "No se puede dividir entre cero" Sino r ← x mod y Si r = 0 entonces escribir "Es divisible" Sino escribir "No es divisible" Fin Si Fin Si Fin

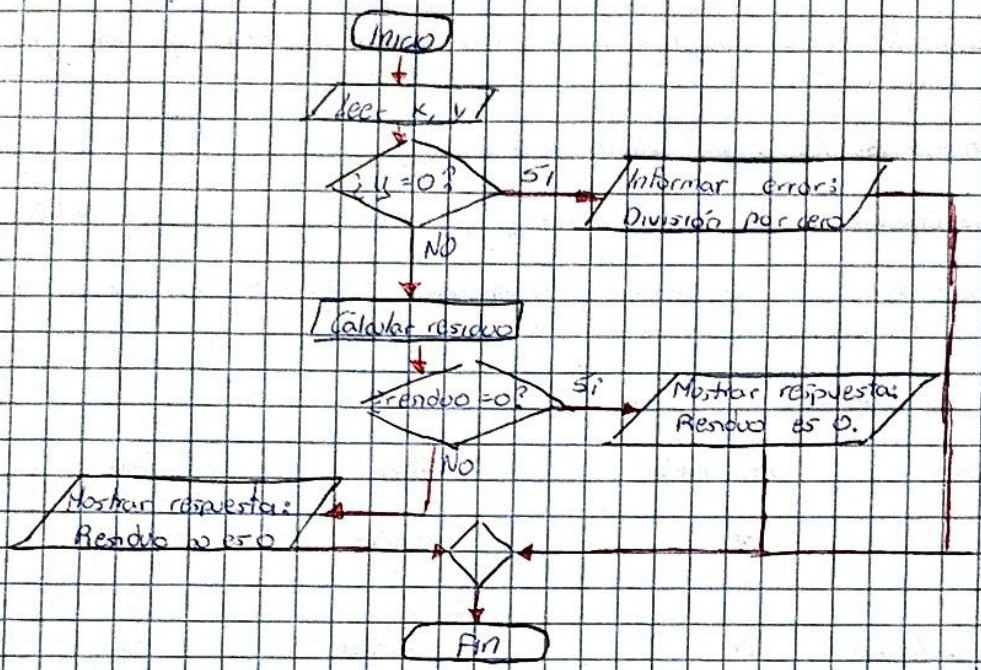
4. Prueba de escritorio.

Entrada / caso	Resultado esperado	
12	4	Divisible
7	2	No divisible
8	0	Divisor inválido

Cortar un árbol es cortar la vida.

5. Diagrama de flujo

Esquema del flujo: Inicio → leer x, y → ¿y=0? → informar error / calcular residuo → ¿residuo=0? → mostrar respuesta → fin



6. Código en C.

```
C
#include <stdio.h>

int main(void) {
    int x, y;

    printf("Ingrese dividendo y divisor: ");
    scanf("%d %d", &x, &y);

    if (y == 0) {
        printf("No se puede dividir entre cero\n");

    } else if (x % y == 0) {
        printf("%d es divisible por %d\n", x, y);

    } else {

    printf("%d no es divisible por %d\n", x, y);

    }

    return 0;
}
```

Cortar un árbol es cortar la vida.

Problema 1.4. Intervalo cerrado [0, 10]

Leer un real y comprobar si se encuentra entre 0 y 10, incluidos ambos extremos.

1. Análisis del problema

Descripción: Se verifica matemáticamente que el dato no sea menor que cero y que tampoco supere diez.

Datos de entrada: Los valores indicados en el enunciado

Resultado esperado: Un valor calculado o mensaje que responda a la cuestión planteada.

Validaciones: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos:

Objeto	Identificador	Clase	Tipo
Dato	x	variable	real
Extremo inferior	0	constante	entero
Extremo superior	10	constante	entero

3. Pseudocódigo

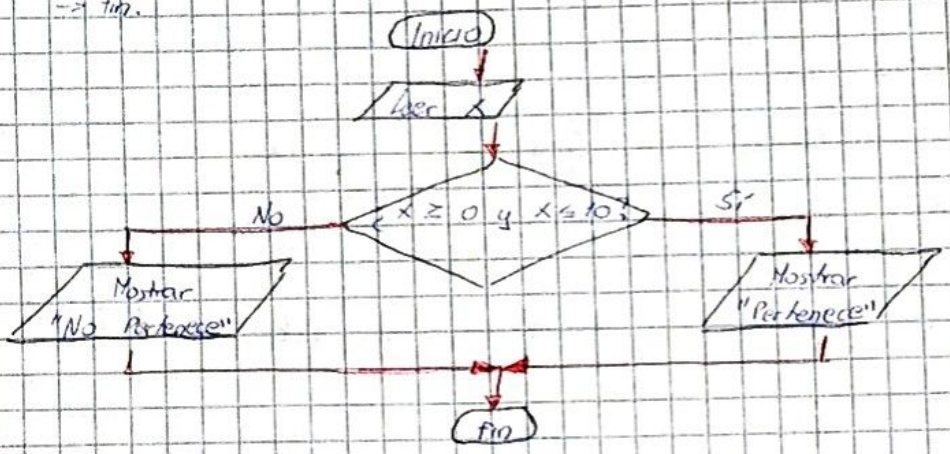
Inicio leer x Si x >= 0 Y x <= 10 entonces Escribir "pertenece" Sino Escribir "No pertenece" Fin Si Fin

4. Prueba de escritorio

Entrada / caso	Resultado esperado
5	SI
0	SI
10	SI
-0,5	NO
12	NO

5. Diagrama de flujo

Esquema del flujo: Inicio → leer x → ¿x ≥ 0 y x ≤ 10? → mostrar pertenece/no pertenece → fin.



6. Código en C

```c
#include <stdio.h>

int main(void) {
    double x;

    printf("Ingrese un número: ");
    scanf("%lf", &x);

    if (x >= 0 && x <= 10) {
        printf("Pertenece al intervalo [0, 10]\n");
    } else {
        printf("No pertenece al intervalo [0, 10]\n");
    }

    return 0;
}
```

Problema 1.5 Conversión de unidades de tiempo.

Convertir una cantidad de segundos en días, horas, minutos y segundos.

1. Análisis del Problema

Descripción: Se extraen primero los días completos. Con el residuo se calculan las horas; el nuevo residuo permite obtener minutos y segundos.

Datos de entrada: los valores indicados en el enunciado.

Resultado esperado: Un valor calculado o un mensaje que responda a la condición planteada.

Validación: Se comprueban los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos

Objeto	Identificador	Clase	Tipo
Segundos iniciales	c	variable	entero
Días	días	variable	entero
Horas	horas	variable	entero
Minutos	minutos	variable	entero
Segundos sobrantes	seg	variable	entero
Segundos por día	86400	constante	entero
Segundos por hora	3600	constante	entero
Segundos por minuto	60	constante	entero

3. Pseudocódigo

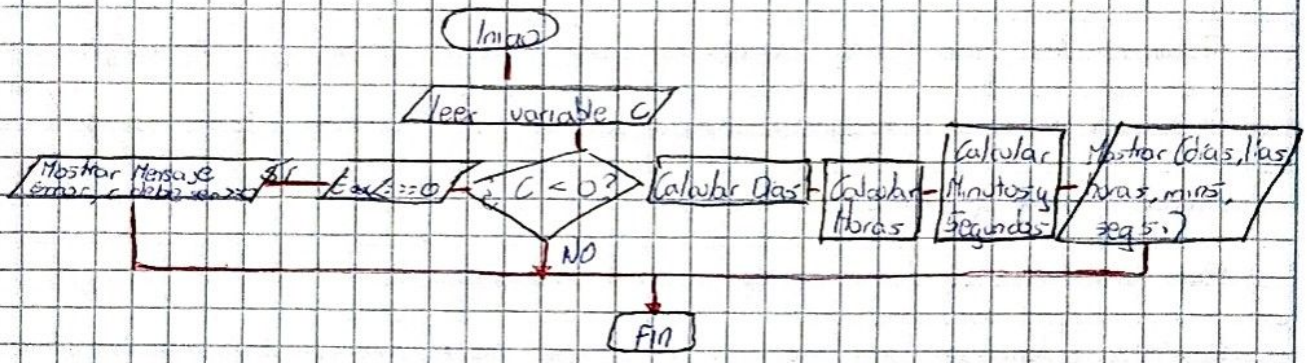
Inicio leer c si c < 0 entonces escribir "Dato no válido" sino días ← c div 86400 resto ← c mod 86400 horas ← resto div 3600 resto ← resto mod 3600 minutos ← resto div 60 seg ← resto mod 60 Escribir días, horas, minutos, seg fin si fin.

4. Prueba de escritorio

Entrada / caso	Resultado esperado
90061 3600 -4	1 día, 1 hora, 1 minuto, 1 segundo 0 días, 1 hora, 0 minutos, 0 segundos Dato no válido.

5) Diagrama de flujo

Esquema del flujo: Inicio -> leer c -> ¿c<0? -> error / calcular días -> calcular horas -> calcular minutos y segundos -> mostrar -> fin.



6. Código en C

```c
#include <stdio.h>

int main(void) {
    int c, dias, horas, minutos, seg, resto;

    printf("Ingrese segundos: ");
    scanf("%d", &c);

    //Validación para evitar valores negativos
    if (c < 0) {
        printf("La cantidad no puede ser negativa\n");
        return 0;
    }
    // Calculos de conversión
    dias = c / 86400;
    resto = c % 86400;

    horas = resto / 3600;
    resto %= 3600;

    minutos = resto / 60;
    seg = resto % 60;

    //Impresión de resultados
    printf("%d días, %d horas, %d minutos y %d segundos

    return 0;

}
```

Problema 1.6 Máximo, mínimo y media de alturas

Leer alturas hasta que se ingrese un valor negativo y calcular media, máximo y mínimo

1. Análisis del problema

Descripción: El valor negativo funciona como señal de terminación y no se incluye.

Se acumula la suma y se cuenta cada altura válida. La primera lectura válida inicializa máximo y mínimo

Datos de entrada. Los valores indicados en el enunciado

Resultados esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto

2. Tabla de objetos

Objeto	Identificador	Clase	Tipo
Altura leída	h	variable	real
Acumulador	suma	variable	real
Cantidad	cont	variable	entero
Mayor altura	maximo	variable	real
Menor altura	minimo	variable	real
Promedio	media	variable	real

3. Pseudocódigo.

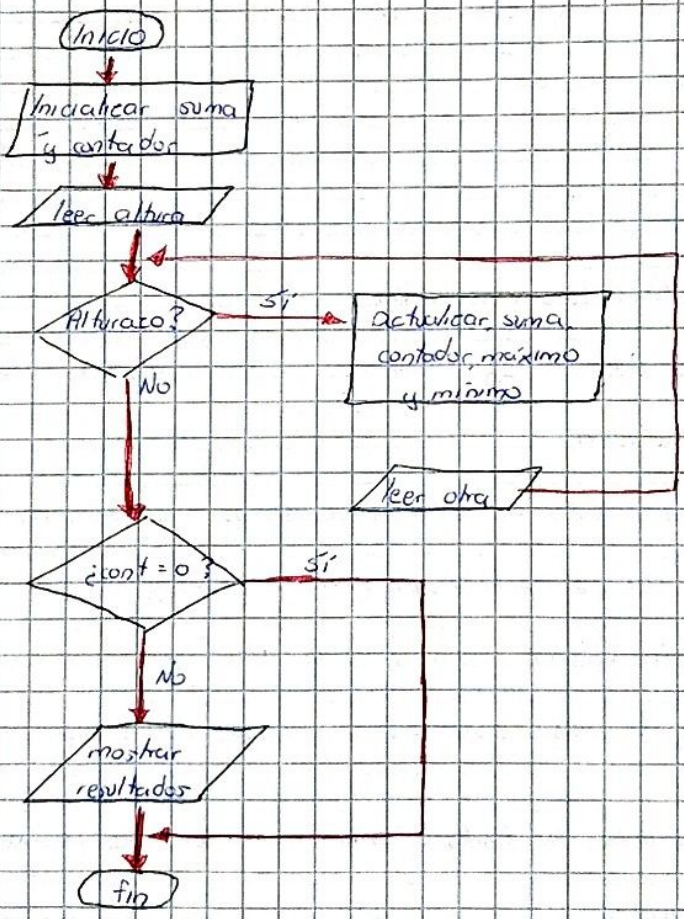
Inicio suma ← 0; cont ← 0 leer h Mientras h >= 0 hacer Si cont = 0 entonces maximo ← h; minimo ← h Sino actualizar maximo y minimo suma ← suma + h; cont ← cont + 1 leer h fin Mientras Si cont = 0 escribir "Sin datos" Sino media ← suma / cont; mostrar media, maximo, minimo fin.

4. Prueba de escritorio

Entrada / caso	Resultado esperado
1.70, 1.65, 1.80, -1	Media 1.72; máx. 1.80; mín. 1.65
1.50, -1	Media, máximo y mínimo 1.50
-1	No se ingresaron alturas.

5. Diagrama de flujo.

Esquema del flujo: Inicio -> inicializar suma y contador -> leer altura -> ¿altura > 0? -> actualizar suma, contador, máximo y mínimo -> leer otra -> al terminar ¿cont > 0? -> mostrar resultados -> fin.



6. Código en C

```c
#include <stdio.h>

int main(void) {
    double h, suma = 0, maximo = 0, minimo = 0, media;
    int cont = 0;
```

```
printf("Ingrese alturas (negativo para terminar): ");scanf
("%f", &h);

while (h >= 0) {
    if(cont == 0){
        maximo = minimo = h;
    } else {
        if (h > maximo) maximo = h;
        if (h < minimo) minimo = h;
    }

    suma += h;
    cont++;
    scanf("%f", &h);

}
if (cont == 0){
    printf("No se ingresaron alturas\n");
} else {
    media = suma / cont;
    printf("Media %.2f, maximo %.2f, minimo %.2f\n"
}
return 0;
}
```

Problema 1.7 Raíz cuadrada entera

Encontrar el mayor entero R cuyo cuadrado no exceda el entero positivo N.

1. Análisis del Problema

Descripción: Se incrementa un contador mientras su cuadrado sea menor o igual que N. Al salir del ciclo, el contador ya superó la raíz y por eso se resta uno.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

Objeto	Identificador	Clase	Tipo
Numerador	N	variable	entero
Contador	i	variable	entero
Raíz entera	R	variable	entero

3. Pseudocodigo

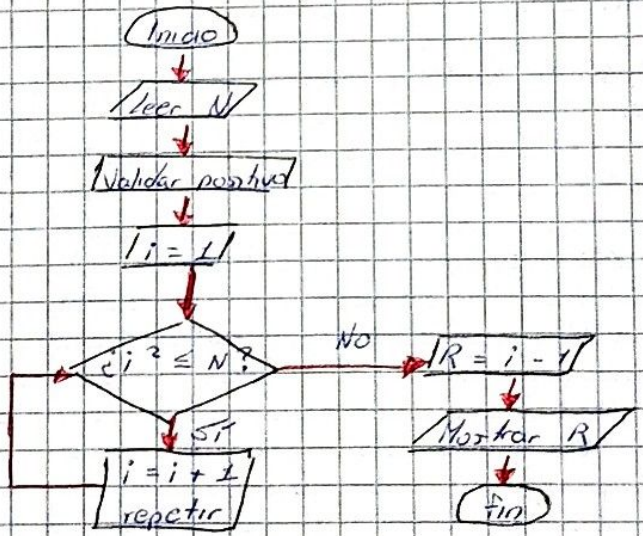
Inicio Leer N si N <= 0 escribir "N debe ser positivo" sino i <- 1 Mientras i*i <= N hacer i <- i + 1 R <- i - 1 Escribir R fin si fin

4. Prueba de escritorio

Entrada / Caso	Resultado esperado
10	3
16	4
2	1
0	Dato invalido

5. Diagrama de flujo

Esquema del flujo: Inicio -> leer N -> validar positivo -> iniciar i = 1 -> ¿$i^2 \leq N$? -> incrementar i y repetir -> R = i - 1 -> mostrar -> fin



Codigo

C

```
#include <stdio.h>

int main (void) {
    long long n, i = 1, r;

    printf("Ingrese un entero positivo:");
    scanf("%lld", &n);
```

```
if (n <= 0) {
    printf("N debe ser positivo\n");
} else {
    while (i * i <= n) {
        i++;
    }
    r = i - 1;
    printf("Raiz entera: %lld\n", r);
}

return 0;
}
```

Problema 1.8 Número primo

Decir si un entero mayor que uno tiene únicamente dos divisores positivos.

1. Análisis del problema

Descripción: Se prueban divisores desde dos. Si alguno divide exactamente al número, este es compuesto. Basta revisar hasta que d por d supere n.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos

Objeto	Identificador	Clase	Tipo
Número	n	variable	entero
Divisor de prueba	d	variable	entero
Indicador	esPrimo	variable	lógico

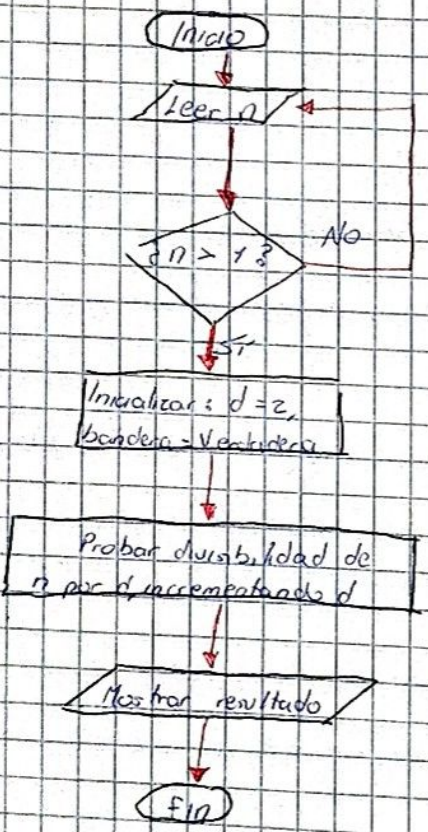
3. Pseudocódigo

Inicio leer n si n <= 1 escribir "Debe ser mayor que 1" Sino esPrimo ← Verdadero, d ← 2 Mientras d * d <= n Y esPrimo hacer si n mod d = 0 entonces esPrimo ← Falso d ← d + 1 fin Mientras Mostrar si esPrimo Fin Si Fin

4. Pruebas de escritorio

Entrada / caso	Resultado esperado
17	Primo
9	No primo
2	Primo
1	Dato inválido

5. Diagrama de flujo: Inicio -> leer n -> validar n > 1 -> d ≤ √n y bandera verdadera -> probar divisibilidad -> mostrar resultado -> fin



6. Código en C

```c
#include <stdio.h>

int main(void) {
    int n, d = 2, primo = 1;
```

```
printf ("Ingrese un entero mayor que 1: ");
scanf ("%d", &n);
if (n <= 1) {
    printf ("El numero debe ser mayor que 1\n");
} else {
    while (d * d <= n && primo) {
        if (n % d == 0) {
            primo = 0;
        }
        d++;
    }
    if (primo) {
        printf ("%d es primo\n", n);
    } else {
        printf ("%d no es primo\n", n);
    }
}
return 0;
}
```

Problema 1.9. Primos comprendidos entre dos números

Mostrar los números primos ubicados entre dos extremos enteros ingresados.

1. Análisis del problema

Descripción: Se ordenan los extremos si llegan invertidos. Después se recorre el intervalo y se aplica a cada número la prueba de primalidad. Los valores menores que dos se descartan.

Datos de entrada: Los valores indicados en el enunciado

Resultado esperado: Un valor calculado o un mensaje que responda a la consulta planteada.

Validación: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de Objetos

Objeto	Identificador	Clase	Tipo
Extremos	a, b	variables	enteros
Auxiliar	aux	variable	entero
Candidato	n	variable	entero
Divisor	d	variable	entero
Bandera	primo	variable	lógico
Cantidad	cont	variable	entero

3. Pseudocódigo

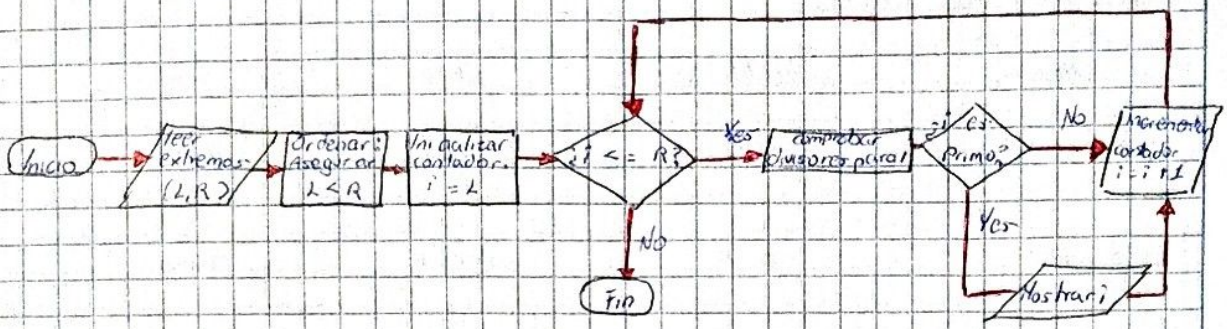
Inicio leer a, b si a>b intercambiar a y b si a<2 entonces a ← 2 cont ← 0 para n desde a hasta b hacer primo ← verdadero, d ← 2 Mientras d * d <= n Y primo hacer si n mod d=0 entonces primo ← falso d ← d+1 fin Mientras si primo entonces mostrar n; cont ← cont + 1 fin Para si cont = 0 mostrar "No hay primos" fin

4. Prueba de escritorio

Entrada / caso	Resultado esperado
10, 30	11, 13, 17, 19, 23, 29
30, 10	11, 13, 17, 19, 23, 29
14, 16	No hay primos

5. Diagrama de flujo

Esquema del flujo: Inicio → leer extremos → ordenar → recorrer cada candidato → comprobar divisores → si es primo mostrarlo → revisar contador → fin.



6. Código en C

```
#include <stdio.h>

int main (void){
    int a, b, aux, n, d, primo, cont = 0;

    printf("Ingrese dos extremos:");
    scanf("%d %d", &a, &b);

    // Si el primer número es mayor, los intercambia
    if (a>b){
        aux = a;
        a = b;
        b = aux;

    }

    // El primer número primo es el 2
    if (a<2) {
        a = 2;
    }

    // Recorre todos los números dentro del rango [a,b]
    for (n = a; n <= b; n++) {

    // Verifica si n es divisible por algún número
    for (d = 2; d*d <= n && primo; d++) {
        if (n%d == 0){
            primo = 0;
        }
    }

    // Si no se encontró ningún primo en el rango
    if (cont==0){
        printf("No hay numeros primos en el rango");

    }
    printf("\n");
    return 0;
}
```

Problema 1, 16, Factorial

Calcular n! para un entero entre cero y veinte, recordando que 0! vale uno

1. Análisis del problema

Descripción: El factorial es el producto de los enteros positivos hasta n. Se comienza en uno; para n igual a cero no se ejecutan multiplicaciones.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validaciones: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos.

Objeto	Identificador	Clase	Tipo
Entrada	n	variable	entero
Producto acumulado	fact	variable	entero
Contador	i	variable	entero
límite	20	constante	entero

3. Pseudocódigo.

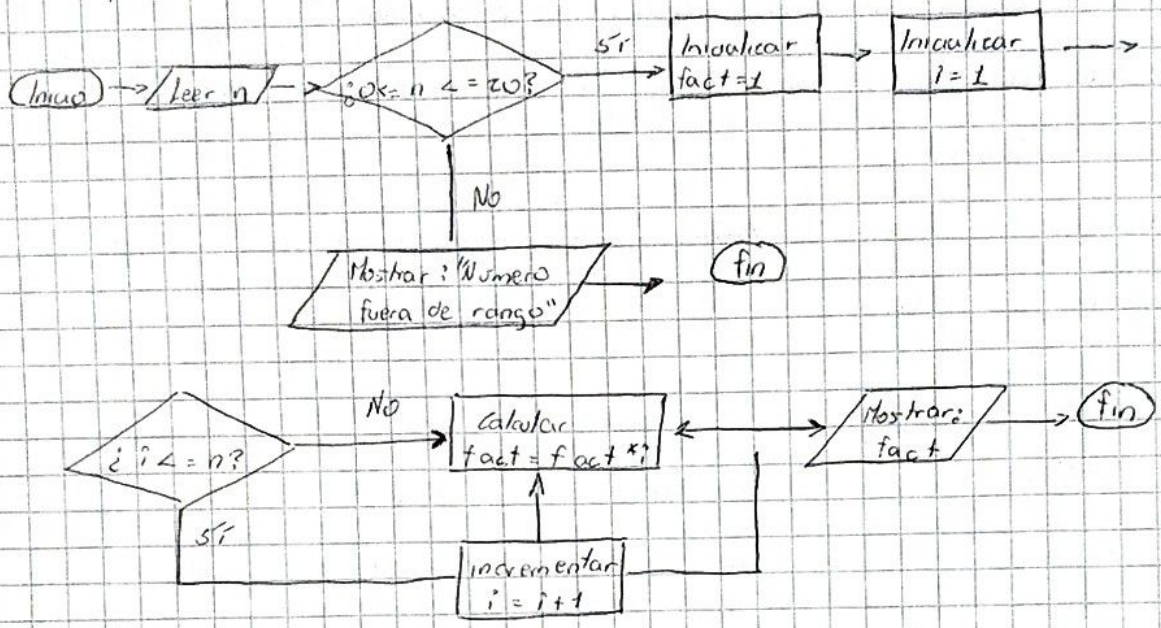
Inicio Leer n Si n<0 o n>20 escribir "Fuera del rango" Sino fact ← 1 Para i ← 1 hasta n hacer fact ← fact * i Escribir fact Fin Si Fin Si

4. Prueba de escritorio.

Entrada / Caso	Resultado esperado
5	120
0	1
10	3628800
21	Fuera del rango.

5. Diagrama de flujo.

Esquema del flujo:



6. Código en C

```
#include <stdio.h>

int main(void) {
    int n, i;
    unsigned long long fact = 1;

    printf("Ingrese n (0 a 20): ");
    scanf("%d", &n);

    if (n < 0 || n > 20) {
        printf("Ingrese un valor entre 0 y 20\n");
    } else {

        for (i = 1; i <= n; i++) {
            fact *= i;

        }
        printf("%d! = %llu\n", n, fact);
    }
    return 0;
}
```

Problema 1.11. Serie de Fibonacci

imprimir los primeros n términos de Fibonacci con n entre uno y noventa.

1. Análisis del problema.

Descripción: se conservan los dos términos anteriores. En cada iteración se muestra el primero, se calcula el siguiente como suma y se desplazan los valores.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos.

Objeto	Identificador	Clase	Tipo
Cantidad de términos	n	variable	entero
Anterior	ant	variable	entero largo
Actual	act	variable	entero largo
Siguiente	sig	variable	entero largo.
Contador.	i	variable.	entero

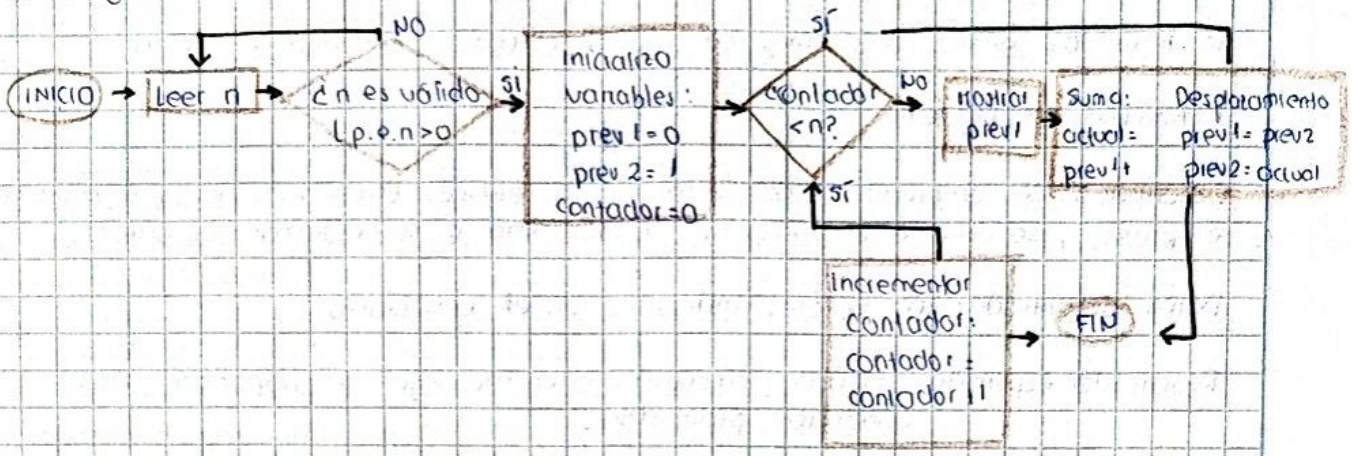
3. Pseudocódigo

inicio leer n Si n<1 o n>90 escribir "cantidad no válida" sino ant←0; act←1 Para i←1 hasta n hacer Escribir ant sig←ant+act; ant←act; act←sig Fin Para Fin Si Fin.

4. Prueba de escritorio.

Entrada / caso	Resultado esperado
7	0 1 1 2 3 5 8
1	0
10	0 1 1 2 3 5 8 13 21 34.

5. Diagrama de flujo



6. Código en C (Code :: Blocks)

```
c
#include <stdio.h>
int main(void){
    int n, i;
    unsigned long long ant = 0, act = 1, sig;
    printf("Cantidad de terminos (1 a 90): ");
    scanf("%d", &n);
if(n<1||n>90){
    printf("Ingrese un valor entre 1 y 90\n");
    }else{
for(i=1; i<=n; i++){
        printf("%llu ", ant);
            sig = ant + act;
            ant = act;
            act = sig;
}printf("\n");} return 0;}
```

Problema 1.12. Suma de dígitos

Obtener la suma de las cifras de un entero positivo

1. Análisis del problema

Descripción: El último dígito se obtiene con el residuo al dividir entre diez. Luego se elimina esa cifra mediante división entera y se repite el proceso.

Datos de entrada: Los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos.

Objeto	Identificador	Clase	Tipo
Número original	n	Variable	entero
Copia de trabajo	copia	variable	entero
Suma acumulada	suma	variable.	entero
Base decimal	10	constante	entero.

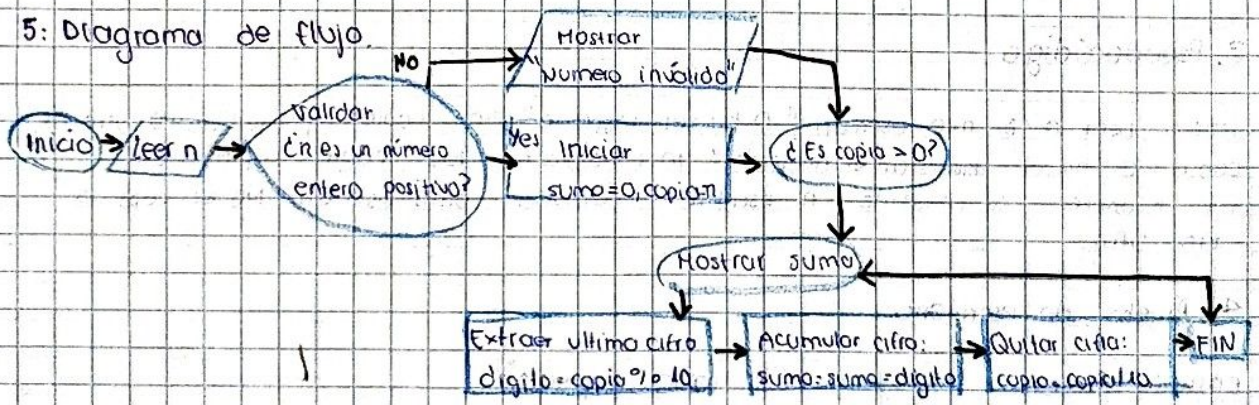
3. Pseudocódigo

inicio Leer n Si n<=0 escribir "Debe ser positivo" Sino copia ← n; suma←0 Mientra copia >0 hacer mod 10) copia ← copia div 10 Fin Mientros Escribir suma Fin SiFin.

4. Prueba de escritorio.

Entrada / caso	Resultado esperado
4825	19
7	7
100	1
0	Dato inválido

5: Diagrama de flujo.



6: Codigo en (Code:: Blocks)

```
#include <stdio.h> int main (void) {
int n, copia, suma = 0;
printf ("Ingrese un entero positivo: ");
scanf ("%d", &n);
if (n <= 0) {
printf ("El numero debe ser positivo\n"); } else {
    copia = n; while (copia > 0) { suma = copia % 10;
        copia /= 10; }
printf ("Suma de cifras:
%d\n", suma ); } return 0; }.
```

Problema 1.13 Número capicúa.

Comprobar si un entero positivo conserva el mismo valor el intervenir el orden de sus cifras.

1. Análisis del problema

Descripción: Se construye el número invertido extrayendo cifras de derecha a izquierdo Al final se compara con el número inicial

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos.

Objeto	Identificador	Clase	Tipo
Número ingresado	n	variable	entero largo
Copia	copia	variable	entero largo
Invertido	invertido	variable	entero largo
Base	10	constante	entero.

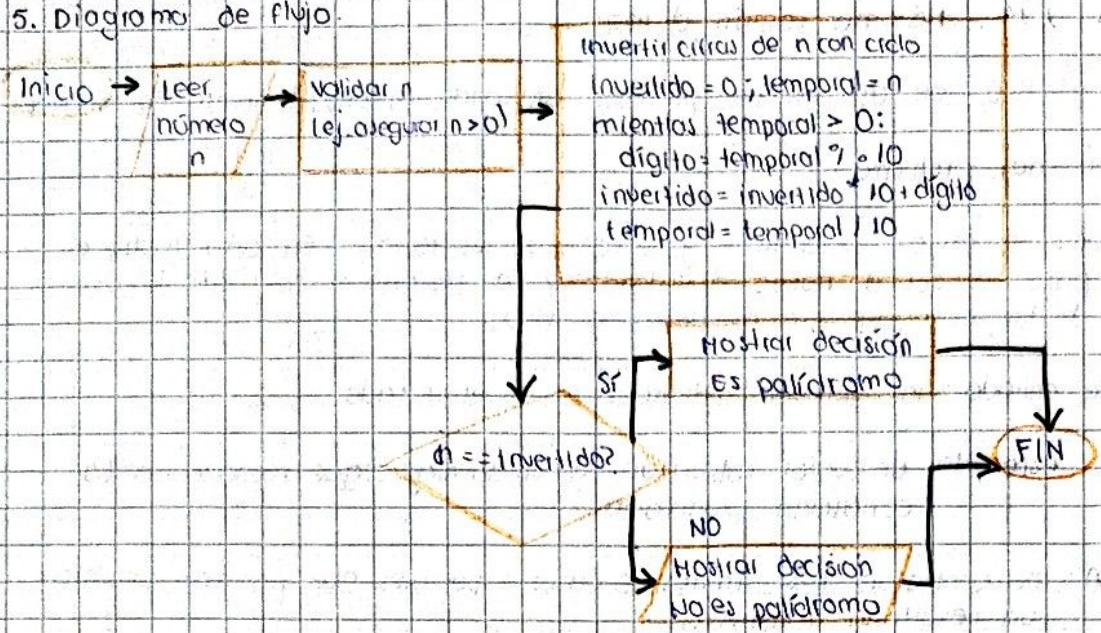
3. Pseudocódigo

Inicio Leer n Si n<0 escribir "Debe ser positivo" sino copia ← n; invertido ← 0 mientras copia > 0 hacer invertido ← invertido * 10 + (copia mod 10) copia ← copia div 10 Fin mientras Si invertido = n escribir "Capicúa sino escribir "NO es capicúa" Fin sifin.

4. Prueba de escritorio

Entrada / caso	Resultado esperado
12321	Capicúa
1231	No es capicúa
1221	Capicúa
100	No es capicúa.

5. Diagrama de flujo



6: Código en C (Code :: Blocks)

```c
#include <stdio.h>
int main(void){
long long n, copia, invertido=0;
printf ("Ingrese un entero positivo: ");
 scanf ("%lld", &n);
if (n<=0){
printf ("El numero debe ser positivo\n");
  } else {
    copia = n;
while (copia>0) {
   invertido = invertido * 10 + copia % 10; copia /= 10;
}
  if (invertido == n) {
    printf ("%lld es capicua\n", n);
} else {
  printf ("%lld no es capicua\n", n);
}
}
return 0;
}
```

Problema 1.14. Máximo común divisor.

Calcular el MCD de dos enteros positivos aplicando el procedimiento de Euclides.

1. Análisis del problema

Descripción: Se divide a entre b y se conserva el residuo. En cada vuelta el divisor pasa a ser el nuevo dividendo y el residuo se convierte en divisor. Cuando b llega a cero, a contiene el MCD.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos.

Objeto	Identificador	Clase	Tipo
Primer entero	a	variable	entero
Segundo entero	b	variable	entero
Residuo	r	variable.	entero

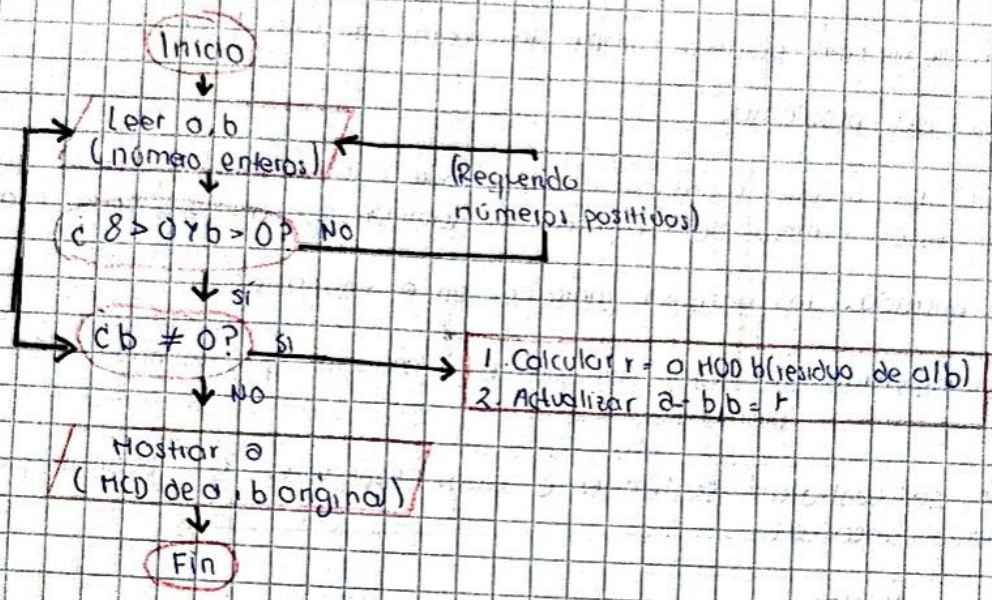
3. Pseudocódigo

inicio leer a,b si a<=0 o b<=0 escribir "Datos no válidos" sino Mientras b<>0 hacer Mientras Escribir a Fin SiFin.

4. Prueba de Escritorio

Entrada / caso	Resultado esperado
48, 18	6
100, 75	25
17, 5	1
10, 10	10

3. Diagrama de flujo



6. Código en C (Code :: Blocks)

```
c
# include <studio.h>
int main (void) {
    int a, b, r;
printf ("Ingrese dos enteros positivos: ");
  scanf ("%od %od", &a, &b);
if (a <0 || b <= 0) {
  printf ("Los dos valores deben ser positivos\n")
} else {
while (b != 0) {
  r = a % b;
   a = b;
   b = r;
}
printf (" MCD = %d\n", a);
}
return 0;
}
```

Problema 1.15. Año bisiesto

Determinar si un año positivo cumple las reglas de año bisiesto.

1. Análisis del problema.

Descripción: Es bisiesto cuando es divisible entre cuatro, salvo los años divisibles entre cien; estos últimos solo clasican si también son divisibles entre cuatrocientos

Datos de entrada. los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos.

Objeto	Identificador	Clase	Tipo
Año	anio	variable	entero
Divisor ordinario	4	constante	entero
Divisor del siglo	100	constante	entero
Excepción	400	constante	entero

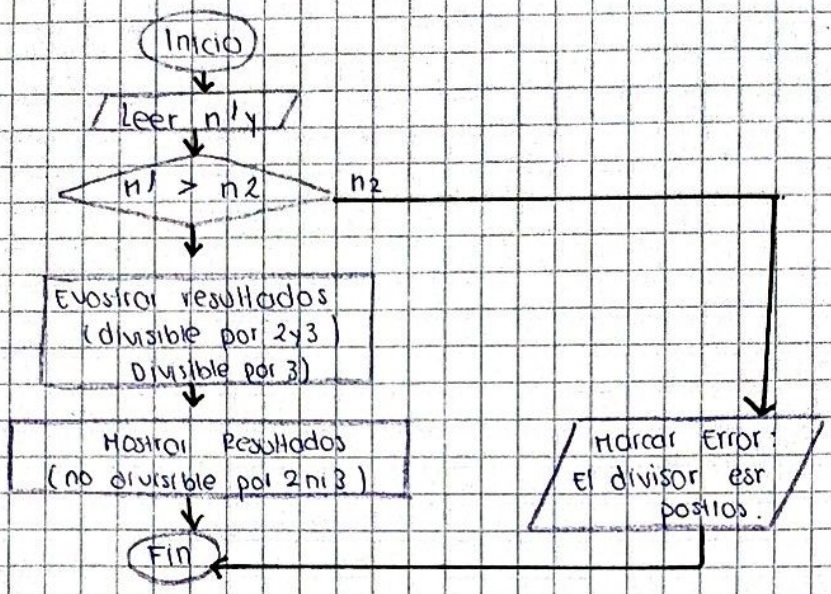
3. Pseudocódigo

Inicio leer anio si anio <=0 escribir "Año invalido" sino si (anio mod 4=0 Y anio mod 100<>0) o anio mod 400=0 escribir "Bisiesto sino escribir "No bisiesto" Fin Sifin

4. Prueba de escritorio

Entrada / caso	Resultado esperado
2024	Bisiesto
2023	No bisiesto
1900	No bisiesto
2000	Bisiesto

5. Diagrama de flujo



6. Código en C (code :: Blocks)

```c
#include <stdio.h>
    int anio;
printf("Ingrese el anio: ");
    scanf("%d", &anio);
if (anio <=0) {
    printf("Anio no valido\n");
}else if ((anio % 4 == 0 && anio % 100 != 0) ||on
    printf("%od es bisiesto\n", anio);
} else {
    printf("%o no es bisiesto\n", anio);
  }
  return 0;
}
```

Problema 1.16 Número perfecto

Verificar si un entero positivo equivale a la suma de sus divisores propios.

1. Análisis del problema

Descripción: Se recorren los enteros menores que n. Cada divisor exacto se añade a un acumulador y al terminar se compara la suma con el número original.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos:

Objeto	Identificador	Clase	Tipo
Número	n	variable	entero
Divisor candidato	d	variable	entero
Suma de divisores	suma	variable	entero

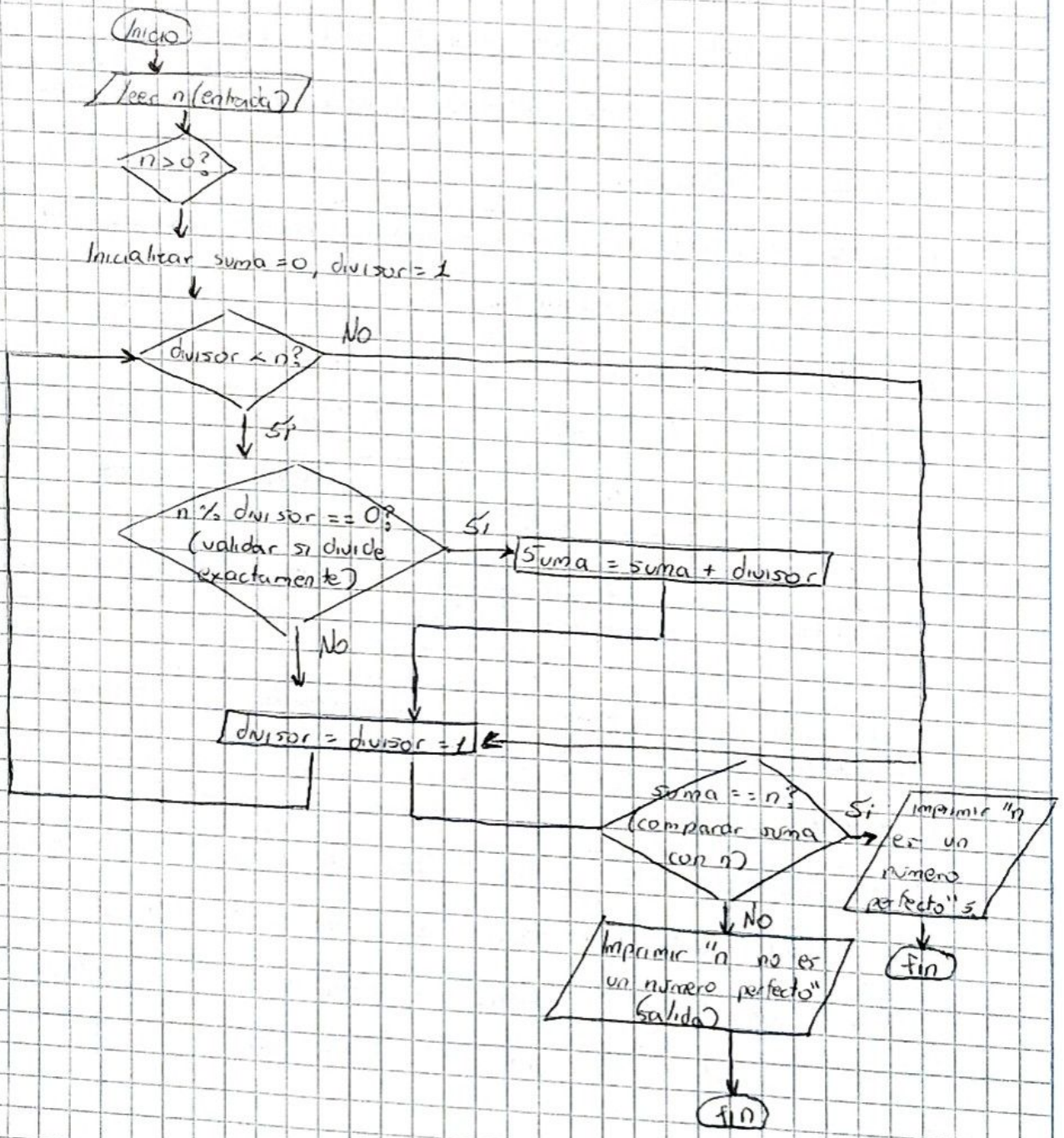
3. Pseudocódigo

Inicio Leer n Si n <= 0 escribir "Debe ser positivo" Sino suma <- 0
Para d <- 1 hasta n-1 hacer Si n mod d = 0 entonces
suma <- suma + d Fin Para Si suma = n escribir "Perfecto"
Sino escribir "No perfecto" Fin Si Fin

4. Prueba de Escritorio

Entrada / caso	Resultado esperado
6	Divisores 1, 2, 3; suma 6; perfecto
28	Suma 28; perfecto
12	Suma 16; no perfecto
1	Suma 0; no perfecto

5. Diagrama de Flujo



Código en C

```
#include <stdio.h>

int main(void) {
    int n, d, suma = 0;

    printf("Ingrese un entero positivo: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("El número debe ser positivo\n");
    } else {
        for (d = 1; d < n; d++) {
            if (n % d == 0) {
                suma += d;
            }
        }

        if (suma == n) {
            printf("%d es perfecto\n", n);
        } else {
            printf("%d no es perfecto (suma = %d)\n", n, suma);
        }
    }
    return 0;
}
```

Problema 1.17. Conversión decimal a binario

Transformar un entero positivo de base diez a su representación binaria.

1. Análisis del problema

Descripción: Se divide sucesivamente el número entre dos. Los residuos obtenidos son los bits desde el último hasta el primero; se acumulan con potencias decimales para mostrarlos en orden correcto, dentro del límite indicado.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos.

Objeto	Identificador	Clase	Tipo
Número decimal	n	variable	entero
Copia	copia	variable	entero
Residuo	resto	variable	entero
Resultado binario	binario	variable	entero largo
Peso decimal	peso	variable	entero largo

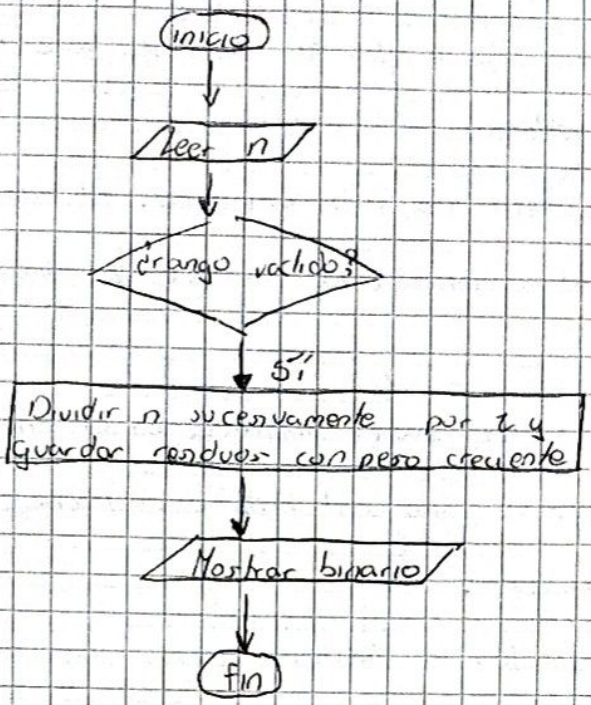
3. Pseudocódigo

Inicio Leer n Si n<1 o n>1 o n>2^63 escribir "Fuera del rango"
Sino copia<-n; binario <- 0; peso <- 1 Mientras copia >0 hacer resto <-
copia mod 2 binario <- binario + resto * peso peso <- peso * 10
copia <- copia div 2 fin Mientras Escribir binario fin Si fin.

4. Prueba de escritorio

Entrada / caso	Resultado esperado
13	1101
8	1000
255	11111111
0	fuera del rango

5. Diagrama de flujo



6. Código en C

```
#include <stdio.h>

int main(void) {
    long long n, copia, binario = 0, peso = 1;
    int resto;

    printf("Ingrese n (1 a 2e2143): ");
    scanf("%lld", &n);

    if (n < 1 || n > 2e2143) {
        printf("Valor fuera del rango permitido\n");
    } else {
        copia = n;
        while (copia > 0) {
            resto = copia % 2;
            binario += resto * peso;
            peso *= 10;
            copia /= 2;
        }
        printf("%lld en binario es %lld\n", n, binario);
    }
    return 0;
}
```

Problema 1. 1º Triángulo de asteriscos

Dibujar un triángulo con n filas, la fila i contiene i asteriscos.

1. Análisis del problema

Descripción: Se utilizan dos ciclos anidados. El ciclo externo controla las filas y el interno imprime la cantidad de símbolos correspondiente a cada fila.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validaciones: Se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos.

Objeto	Identificador	Clase	Tipo
Filas solicitadas	n	variable	entero
Fila actual	i	variable	entero
Símbolos por fila	j	variable	entero
Asterisco	*	constante	carácter

3. Pseudocódigo

Inicio Leer n Si n<1 escribir "Ingrese al menos una fila" Sino Para i←1 hasta n hacer Para j←1 hasta i hacer escribir "*" Escribir salto de línea Fin Para Fin Si Fin.

4. Prueba de Escritorio

Entrada / caso

4

1

0

Resultado esperado

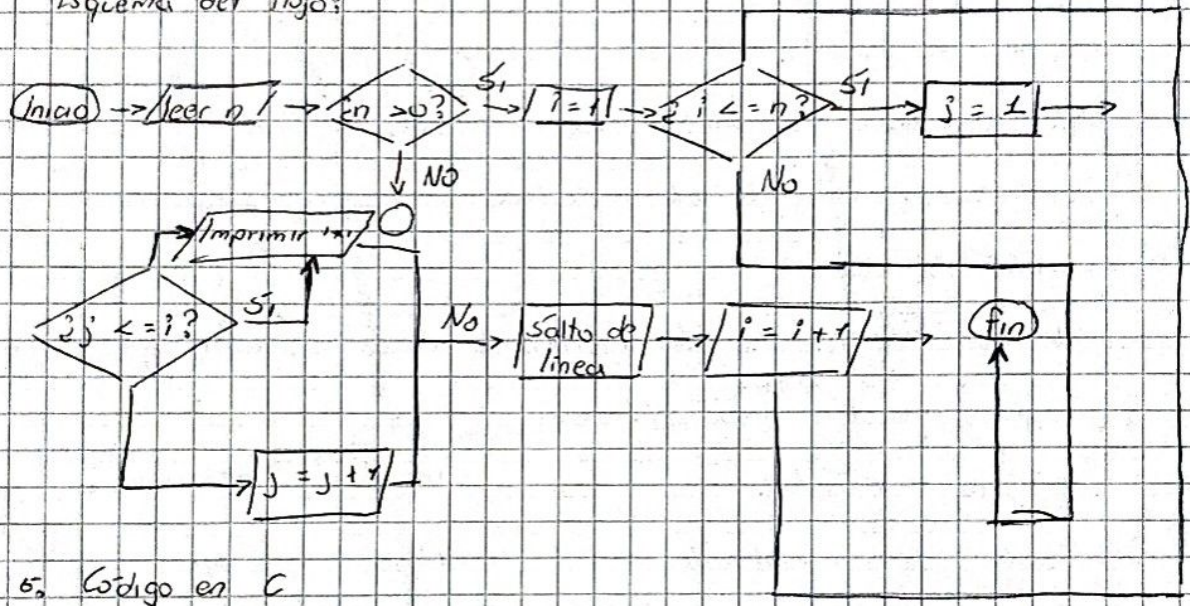
*, **, ***, ****

*

Dato invalido

5. Diagrama de flujo

Esquema del flujo:



6. Código en C

```
#include <stdio.h>

int main(void) {
    int n, i, j;

    printf("Numero de filas: ");
    scanf("%d", &n);

    if(n<1) {
        printf("Ingrese un numero mayor o igual que 1\n");
    } else {
        for(i = 1; i <= n; i++) {
            for(j = 1; j <= i; j++){
                printf("*");
            }
            printf("\n");
        }
    }
    return 0;
}
```

Problema 1, 19 segundo mayor de una secuencia.

Leer enteros positivos hasta recibir cero e informar el mayor y el segundo mayor, si hay menos de dos datos, avisarlo.

1. Análisis del problema

Descripción: Se mantiene en memoria el mayor y el segundo mayor. Cuando aparece un valor que supera al mayor, el anterior mayor pasa a la segunda posición. El cero termina la entrada y los negativos se ignoran.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: Un valor calculado o un mensaje que responda a la condición planteada.

Validación: Se contemplan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de Objetos:

Objeto	Identificador	Clase	Tipo
Dato actual	x	variable	entero
Mayor	mayor	variable	entero
Segundo mayor	segundo	variable	entero
Cantidad válida	cont	variable	entero
Centinela	0	constante	entero

3. Pseudocódigo.

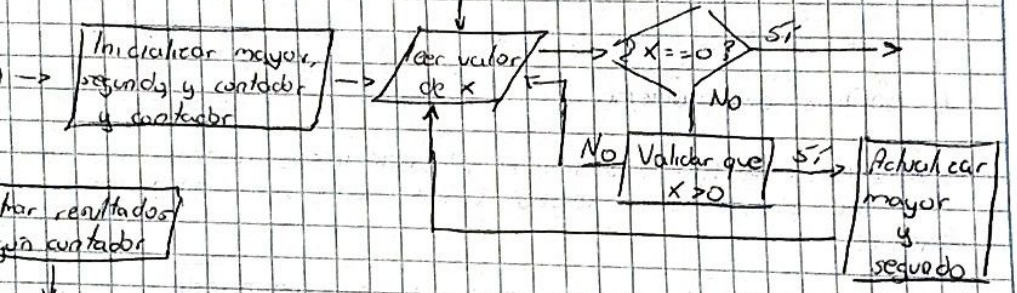
Inicio mayor ← 0 ; segundo ← 0, cont ← 0 leer x Mientras x <> 0 hacer

Si x < 0 escribir "Negativo ignorado" Sino cont ← cont + 1 Si

yor entonces: segundo ← mayor ← x sino si x > segundo entonces:
← x fin si, Leer x fin Mientras si cont < 2 escribir "se
n dos valores" sino mostrar mayor y segundo fin
ca de escritorio.

da / caso	Resultado esperado
3, 9, 5, 0	Mayor 9; segundo 7
9, 5, 0	Mayor 9; segundo 9
1, 0	faltan datos.

agrama de flujo.



ódigo en C.

```
#include <stdio.h>

int main(void) {
    int x, mayor = 0, segundo = 0, cont = 0;

    printf("Ingrese enteros positivos (0 termina)\n");
    scanf("%d", &x);

    while (x != 0) {
        if (x < 0) {
            printf("Valor negativo ignorado\n");
        } else {
            cont++;
            if (x > mayor) {
                segundo = mayor;
                mayor = x;
```

```
    } else if (x > segundo) {
        segundo = x;
    }
}
scanf("%d", &x);
}
if (cont < 2) {
    printf("Se necesitan al menos dos valores\n");
} else {
    printf("Mayor: %d, segundo mayor: %d\n", mayor, segundo);
}
return 0;
}
```

Problema 1.20 Adivina el número

El programa conserva un número secreto entre uno y cien. El usuario prueba hasta acertar y recibe pistas en cada intento.

1. Análisis del problema.

Descripción: Se fija el secreto en 37. Cada intento aumenta el contador y se compara con el valor oculto. El ciclo concluye cuando ambos números coinciden.

Datos de entrada: los valores indicados en el enunciado

Resultado esperado: un valor calculado o un mensaje que responda a la condición planteada.

Validación: se controlan los límites o casos especiales que podrían producir un resultado incorrecto.

2. Tabla de objetos.

Objeto	Identificador	Clase	Tipo
Número reservado	secreto = 37	constante	entero
Intento	intento	variable	entero
Cantidad de intentos	cont	variable	entero

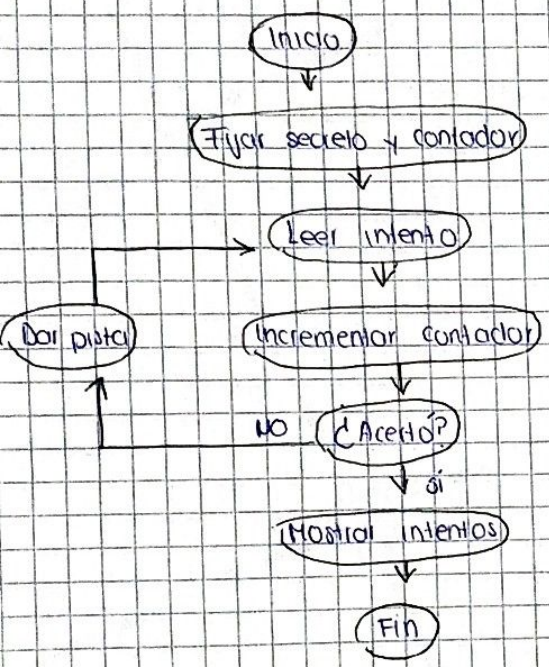
3: Pseudocódigo

Inicio secreto ← 37; cont ← 0 Repetir Leer intento; cont ← cont + 1 Si intento < secreto escribir "Más alto" sino si intento > secreto escribir "Más bajo" sino escribir "Acertaste" Hasta que

4. Prueba de escritorio

Entrada / caso	Resultado esperado
37	Acertó en 1 intento
50, 25, 37	Más bajo; más alto; acertó en 3 intentos
10, 90, 40, 35, 37	5 intentos.

5. Diagrama de flujo



6. Código en C (Code::Blocks)

```c
#include <stdio.h>
int main(void){
    const int secreto = 37;
    int intento, cont = 0;
    do {
        printf("Adivine el número entre 1 y 100: ");
        scanf("%d", &intento);
        cont++;
        if (intento < secreto) {
            printf("Más alto\n");
        } else if (intento > secreto) {
            printf("Más bajo\n");
        } else {
            printf("Acertaste\n");
        } while (intento != secreto);
    printf("Intentos utilizados: %d\n", co
    return 0;
}
```